Error Messages as Documentation

Making Compiler Diagnostics Actionable and Educational

Lateralus Language

bad-antics · April 2026 · Lateralus Language Research

ABSTRACT Lateralus Language -- lateralus.dev

0. Introduction to Error Messages

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
```

```
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

1. Language Server Protocol (LSP) Overview

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,

    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

2. LSP: textDocument/completion

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls

// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}
```

```
// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

3. LSP: textDocument/hover

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific

conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

4. LSP: textDocument/definition

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where

each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

5. LSP: textDocument/references

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe

blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
```

```
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

6. LSP: textDocument/rename

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The

bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

7. LSP: textDocument/formatting

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts,

structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

8. LSP: textDocument/diagnostics

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe

blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
```

```
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

9. LSP: textDocument/codeAction

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that

provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

10. LSP: textDocument/semanticTokens

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language

users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

11. LSP: textDocument/inlayHints

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

12. LSP Server Implementation in Lateralus

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with

links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local
```

```
// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}
```

```
// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

13. LSP: Document Manager

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
```

```
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

14. LSP: Incremental Sync

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed.

Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

15. Error Message Design Principles

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source

breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })

    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

16. Error Codes and Documentation

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for

building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
```

```
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

17. Source Location and Context Display

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples

language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP

server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

18. Did-You-Mean Suggestions

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists

become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

19. Error Recovery Strategies

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a

source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
```

```
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

20. Warning vs. Error Classification

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types

for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes

used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

21. Standard Library Philosophy

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect

millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

22. stdlib: Collections (list, map, set)

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect

millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on

public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local
```

```
// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}
```

```
// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

23. stdlib: String Processing

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically.

Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

24. stdlib: I/O and File System

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

25. stdlib: Networking

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety

system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })

}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

26. stdlib: Cryptography

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int

//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

27. stdlib: Math and Statistics

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation.

Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
```

```
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

28. stdlib: Serialization (JSON, CBOR)

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language

users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

29. stdlib: Concurrency Primitives

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in

Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and

ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

30. stdlib: Testing Framework

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project

manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
```

```
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

31. stdlib: Time and Date

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and

hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum,

constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

32. Package Manager Design

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and

hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

33. Package: Dependency Resolution

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
```

```
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

34. Package: Reproducible Builds

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The

entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs),

watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

35. Package: Capability Attestation

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look

identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

36. Package: Registry Protocol

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
```

```
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

37. Package: Lock Files

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local
```

```
// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}
```

```
// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

38. Package: Versioning Strategy

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:

///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

39. Documentation Generation

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression
```

```
fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

40. Doc Comments: /// Syntax

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The

package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

41. Doctest: Executable Examples

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string

indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The

entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

42. API Stability Guarantees

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

43. Formatter (ltlfmt) Design

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter

enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

44. Formatter: Canonical Rules

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50

lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

45. Formatter: Performance

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for

language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
```

```
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

46. Linter (ltllint) Design

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing

teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

47. Linter: Rule Implementation

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
```

```
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

48. Linter: False Positive Rate

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

49. Debug Adapter Protocol (DAP)

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file,

and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

50. DAP: Breakpoints

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
```

```
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

51. DAP: Variable Inspection

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and

produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch

//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

52. DAP: Stack Traces

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50

lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

53. REPL Implementation

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of

the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
```

```
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

54. REPL: Multi-line Input

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional,

exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection.

The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

55. REPL: Tab Completion

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter

rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL

maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

56. REPL: History and Persistence

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL

maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

57. Polyglot Bridge: Python

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each

expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus

provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

58. Polyglot Bridge: C FFI

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL

maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Implementation Details

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
```

```
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

59. Polyglot Bridge: JavaScript/WASM

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

Implementation Details

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

60. VS Code Extension

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's

stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Implementation Details

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

61. Neovim Integration

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Implementation Details

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
```

```
    code:       int,
    message:    str,
    span:       Span,

    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

62. Emacs Integration

The standard library is the most-used part of any programming language. Its design decisions -- whether to use checked exceptions or Result types, whether iterators are lazy or eager, whether string indices are bytes or codepoints -- affect millions of lines of code written by library users. Lateralus's stdlib design prioritizes correctness (Result and Option types for fallible operations, no null references), performance (lazy iterators by default, zero-allocation hot paths), and ergonomics (method chaining via |>, comprehensive collection APIs). The stdlib is designed to be learned incrementally: a programmer can write useful code with just lists, maps, and println, then discover richer features as needed.

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

Implementation Details

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically.

Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions

// Rule 5: import ordering: stdlib, third-party, local

// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}

// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

63. CI/CD Integration

Package management is the infrastructure that allows software to be composed from independently developed libraries. Lateralus's package manager (ltlpkg) uses a content-addressed store: packages are identified by the SHA-256 hash of their source contents, ensuring reproducible builds. Dependencies are specified by name and version range in a project manifest; the solver finds a consistent set of versions satisfying all constraints. Lock files record the exact versions and hashes used in a build, guaranteeing that all developers and CI systems use identical dependency trees. The package registry enforces capability attestation: every published package must declare its required capabilities.

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum,

constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

Implementation Details

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

```
// Doctest: example in a doc comment is a runnable test
/// Computes the nth Fibonacci number.
/// Example:
///   assert fib(0) == 0
///   assert fib(1) == 1
///   assert fib(10) == 55
fn fib(n: int) -> int {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
// 'ltl test --doctest' extracts and runs these automatically
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

64. Benchmarking Infrastructure

Documentation generation extracts structured documentation from Lateralus source files. Doc comments (lines starting with ///) are associated with the following declaration (function, struct, enum, constant) and included in the compiled output. The doc generator reads the compiled output and produces HTML, PDF, and man page documentation. Type signatures are automatically formatted with links to referenced types. Examples in doc comments (marked with '///') are extracted and run as doctests during CI, ensuring that documented examples remain correct as the API evolves. The entire Lateralus standard library is documented with doc comments and tested via doctests.

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

Implementation Details

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where

each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

```
// Polyglot bridge: call Python from Lateralus
from python import numpy as np
from python import sklearn.linear_model import LinearRegression

fn fit_linear_model(x: list[float], y: list[float]) -> (float, float) {
    let X = np.array(x).reshape(-1, 1)  // Python ndarray
    let Y = np.array(y)
    let model = LinearRegression().fit(X, Y)
```

```
    return (model.coef_[0] as float, model.intercept_ as float)
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

65. Performance Regression Detection

The Lateralus formatter (ltlfmt) enforces a canonical code style, eliminating debates about indentation width, brace placement, and import ordering. The formatter is opinionated: there is one correct formatting for any syntactically valid Lateralus program, and the formatter applies it deterministically. Unlike optional style guides, a canonical formatter enforces consistency across all code in a project -- contributed code, reviewed code, and auto-generated code all look identical. The formatter integrates with the LSP server, running on save in IDEs. It is also integrated into CI, rejecting pull requests that contain unformatted code.

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

Implementation Details

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type

inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe

blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

```
// LSP server: handle textDocument/completion
fn handle_completion(req: &LspRequest) -> LspResponse {
    let params = req.params.as_completion_params()
    let doc = DOC_STORE.get(params.text_document.uri)
    let pos = params.position
    let completions = get_completions(doc, pos.line, pos.character)
    return LspResponse {
        id: req.id,
        result: completions |> map(fn(c) { c.to_lsp_item() })
    }
}

fn get_completions(doc: &Document, line: int, col: int) -> list[Completion] {
    let scope = doc.scope_at(line, col)
    return scope.all_names()
        |> filter(fn(n) { n.starts_with(doc.prefix_at(col)) })
        |> map(fn(n) { Completion { label: n, kind: scope.kind_of(n) } })
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

66. Conclusion

The Lateralus linter (ltllint) performs semantic checks beyond what the type checker requires. It detects: unused variables and imports (the type checker allows them, the linter flags them), functions that are too long (more than 50 lines), deeply nested control flow (more than 4 levels), missing doc comments on public API functions, inefficient patterns (map followed immediately by filter, which could be a single filtered_map), and potential security issues (use of the unsafe FFI without a safety comment). Linter rules are configurable via a project-level configuration file, allowing teams to enforce project-specific conventions.

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and

stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

Implementation Details

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The

bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts, structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

```
// Error message with context and suggestion
// E0312: type mismatch
//
//   12 | let x: int = "hello"
//              ^^^   ^^^^^^^
//              |     found: str
//              expected: int
//
// Suggestion: use int.parse("hello") to convert str to int
struct TypeError {
    code:       int,
    message:    str,
    span:       Span,
    expected:   Type,
    found:      Type,
    suggestion: Option[str],
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

67. Appendix: CLI Reference

The Debug Adapter Protocol (DAP) defines a standardized interface between debuggers and editors, similar to LSP for language intelligence. Lateralus's DAP server supports: breakpoints (line, conditional, exception), step over/into/out, variable inspection (with structured display for lists, maps, and structs), watch expressions, and stack frame inspection. The DAP server communicates with a Lateralus VM modified to support debugging: the VM can pause execution at any instruction, inspect register and stack state, and resume single instruction at a time. The entire Lateralus debugging stack -- from source breakpoint to VM pause to editor display -- is implemented in Lateralus.

The Lateralus REPL (Read-Eval-Print Loop) provides interactive exploration of the language and standard library. Each expression entered at the prompt is compiled, evaluated, and its result printed. Unlike simple script execution, the REPL maintains state across entries: variables defined in one entry are available in subsequent entries. The REPL supports multi-line input (for function definitions and control flow), history navigation, tab completion (using the LSP completion implementation), and syntax highlighting. An enhanced REPL mode provides a structured notebook interface where each cell is independently editable and re-executable, similar to Jupyter but for Lateralus.

The Polyglot Bridge enables Lateralus programs to call and be called from Python, C, and JavaScript. From Lateralus, Python libraries are imported with 'from python import numpy' and called with full type inference for return types. The bridge automatically converts between Lateralus and Python types: lists become Python lists, maps become dicts,

structs become dataclasses. For C interop, Lateralus provides a foreign function interface (FFI) with explicit unsafe blocks that bypass the memory safety system for the duration of the C call. For JavaScript/WASM, Lateralus compiles to WebAssembly and exports functions with typed JavaScript bindings, enabling Lateralus code to run in browsers and Node.js.

Implementation Details

The Language Server Protocol (LSP) defines a JSON-RPC interface between editors and language-specific servers that provide intelligence: completion, hover documentation, go-to-definition, find-all-references, rename, formatting, and diagnostics. By standardizing this interface, LSP decouples language intelligence from editor implementation: one Lateralus LSP server works with VS Code, Neovim, Emacs, Zed, and any other LSP-compatible editor. Lateralus's LSP server is implemented in Lateralus itself, serving as both a practical tool and a demonstration of Lateralus's suitability for building production servers.

Error messages are the primary user interface of a compiler. When code does not type-check, the programmer receives an error message that must convey: what went wrong, where in the source file, and ideally how to fix it. Poorly-designed error messages are one of the top frustrations of new language users, and significantly impact adoption. Lateralus invests heavily in error message quality. Every error type has a unique code (E0001-E9999), a textual description, a source location with underlined context, and where possible a 'did you mean' suggestion or a link to documentation. Error messages are tested as rigorously as language features, with golden files that verify exact output.

```
// ltlfmt: canonical formatting rules
// Rule 1: indent with 4 spaces
// Rule 2: opening brace on same line as fn/if/for
```

```
// Rule 3: max line length 100 chars; wrap long calls
// Rule 4: blank line between top-level definitions
// Rule 5: import ordering: stdlib, third-party, local
```

```
// BEFORE ltlfmt:
fn foo(a:int,b:int)->int{return a+b}
```

```
// AFTER ltlfmt:
fn foo(a: int, b: int) -> int {
    return a + b
}
```

Tool Command LSP Incremental Config

Compiler ltl build via lsp yes ltl.toml

Type checker ltl check diagnostics yes ltl.toml

Formatter ltlfmt format-on-saven/a ltlfmt.toml

Linter ltllint diagnostics yes ltllint.toml

Test runner ltl test inline results yes ltl.toml

Doc generator ltl doc hover docs yes ltl.toml

Package mgr ltlpkg n/a n/a ltl.toml

Lateralus is an open-source, zero-dependency programming language. Project home: https://lateralus.dev. Source: github.com/bad-antics/lateralus-lang. Released under CC BY 4.0.